

A decorative graphic on the left side of the slide, consisting of a network of white lines and small circles on a dark blue background, resembling a circuit board or a tree structure.

GREEDY ALGORITHMS: ACTIVITY SELECTION

DATA STRUCTURES AND ALGORITHMS 2
MARK FLORYAN AND AARON BLOOMFIELD

CLRS READINGS

- Chapter 16, Greedy Algorithms
 - Intro, page 414
 - Section 16.1, Activity Selection

A decorative graphic on the left side of the slide, consisting of a network of thin, light blue lines and small circles, resembling a circuit board or a neural network diagram. The lines are vertical and horizontal, with some diagonal connections, and the circles are placed at various points along these lines.

ACTIVITY SELECTION

ACTIVITY-SELECTION PROBLEM

Activity Selection: Given a list of activities $A = \{a_1, a_2, \dots, a_n\}$ where each activity $a_i = (s_i, f_i)$ is defined by a **start time** and **finish time**. Select the maximum number of activities that can be fit into one schedule

We just want the most activities possible, they are all equally “good”

This problem is sometimes called Interval Scheduling

THE ACTIVITIES!

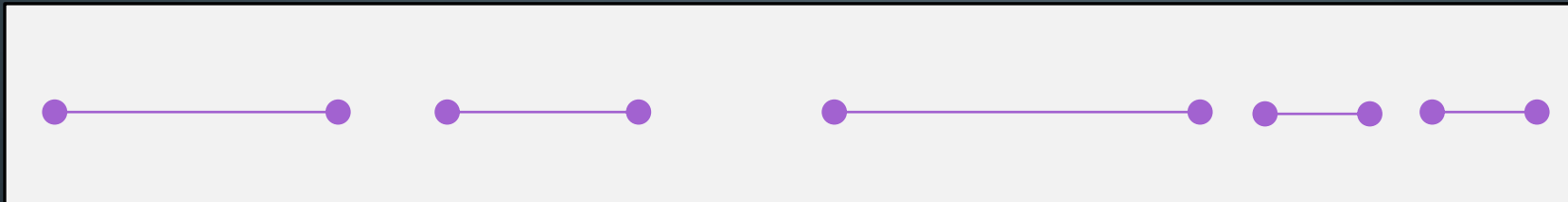
Id	Start	Finish	Activity
1	9:00	10:45	Fractals, Recursion and Crayolas
2	9:15	10:15	Tropical Drink Engineering with Prof. Bloomfield
3	9:30	12:30	Managing Keyboard Fatigue with Swedish Massage
4	9:45	10:30	Applied ChemE: Suntan Oil or Lotion?
5	9:45	11:15	Optimization, Greedy Algorithms, and the Buffet Line
6	10:15	11:00	Hydrodynamics and Surfing
7	10:15	11:30	Computational Genetics and Infectious Diseases
8	10:30	11:45	Turing Award Speech Karaoke
9	11:00	12:00	Pool Tanning for Engineers
10	11:00	12:15	Mechanics, Dynamics and Shuffleboard Physics
11	12:00	12:45	Discrete Math Applications in Gambling

GENERALIZING START, END

Id	Start	Finish	Len	Activity
1	0	6	7	Fractals, Recursion and Crayolas
2	1	4	4	Tropical Drink Engineering with Prof. Bloomfield
3	2	13	12	Managing Keyboard Fatigue with Swedish Massage
4	3	5	3	Applied ChemE: Suntan Oil or Lotion?
5	3	8	6	Optimization, Greedy Algorithms, and the Buffet Line
6	5	7	3	Hydrodynamics and Surfing
7	5	9	5	Computational Genetics and Infectious Diseases
8	6	10	5	Turing Award Speech Karaoke
9	8	11	4	Pool Tanning for Engineers
10	8	12	5	Mechanics, Dynamics and Shuffleboard Physics
11	12	14	3	Discrete Math Applications in Gambling

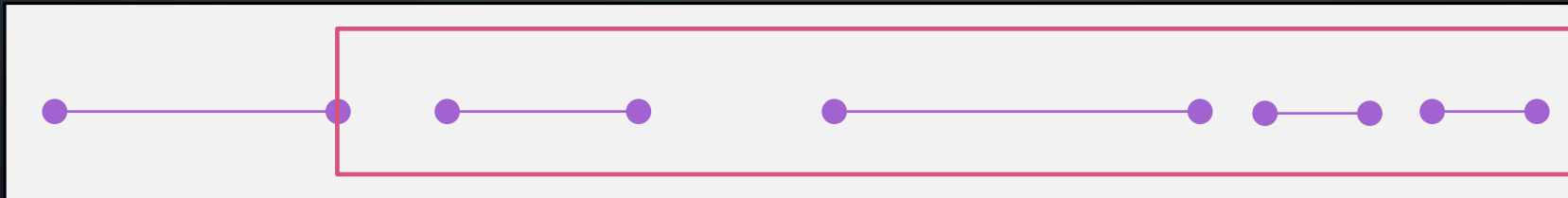
OPTIMAL SUBSTRUCTURE

Does this problem have optimal substructure? Yes, but we need to think about the exact nature of the subproblems



If this is optimal overall

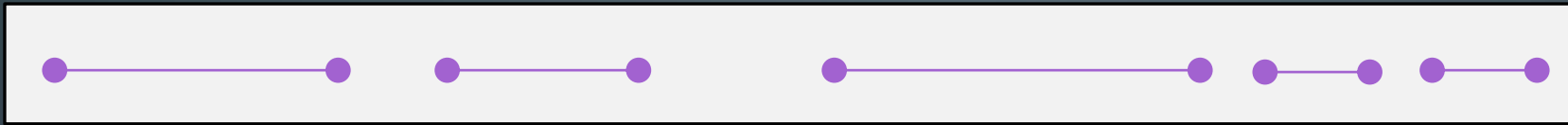
5 activities!



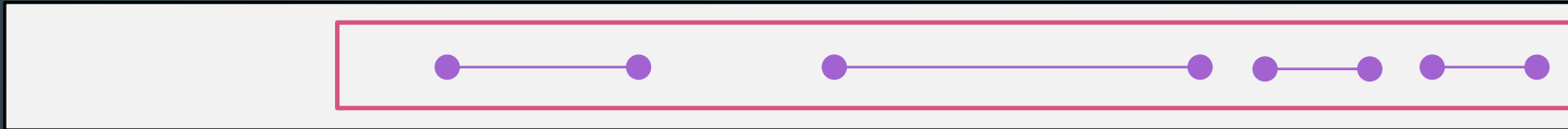
Then is this optimal too?

**Note that subproblem ONLY contains intervals that start after finish time of first activity

OPTIMAL SUBSTRUCTURE PROOF



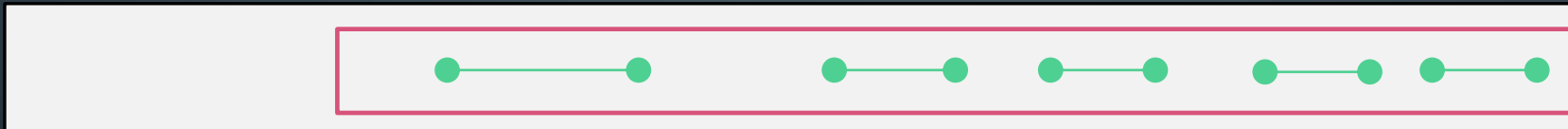
If this is optimal...



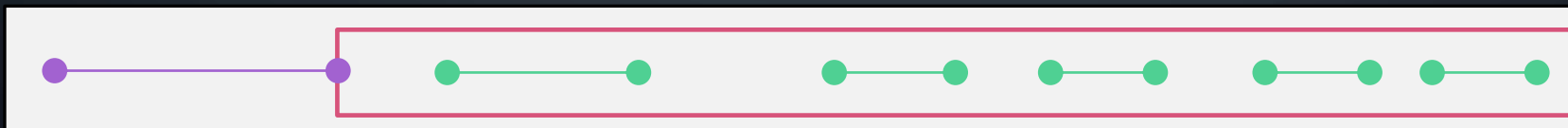
But this is NOT



Then there is a better solution
for this time frame



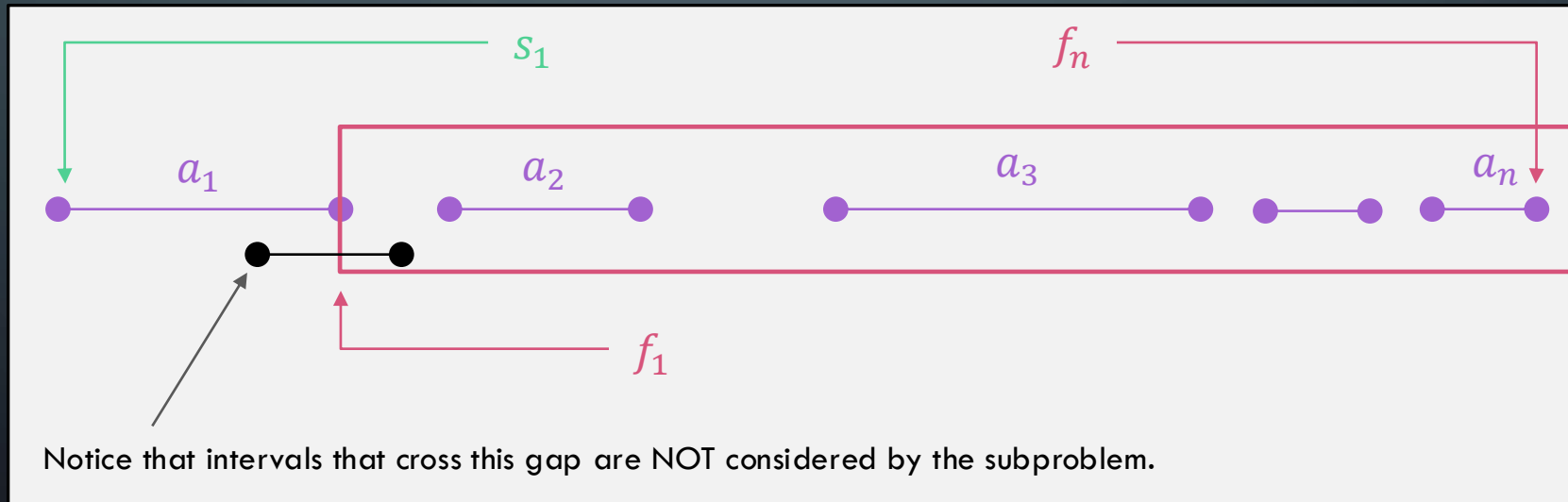
Which can be recombined with first activity to produce
better than optimal solution. Contradiction!



Q.E.D. Problem has
optimal substructure!

OPTIMAL SUBSTRUCTURE

If $A = \{a_1, a_2, \dots, a_n\}$ is the optimal solution to some activity selection problem within the time frame (s_1, f_n) , then $A' = A - \{a_1\} = \{a_2, a_3, \dots, a_n\}$ is the optimal solution within time frame (f_1, f_n)



GREEDY APPROACH

ActivitySelect (Activities) :

Select some first *activity* to schedule

Eliminate *activities* incompatible selected one

Repeat until no *activity* left

Hard Part: *What is a good greedy choice for selecting next item?*

SOME POSSIBLE GREEDY CHOICES

Pick the next compatible activity that **starts earliest**.

Pick the activity that has the **least conflicts** (i.e. overlaps).

Pick the **shortest** activity first.

Pick the next compatible activity that **finishes the earliest**.

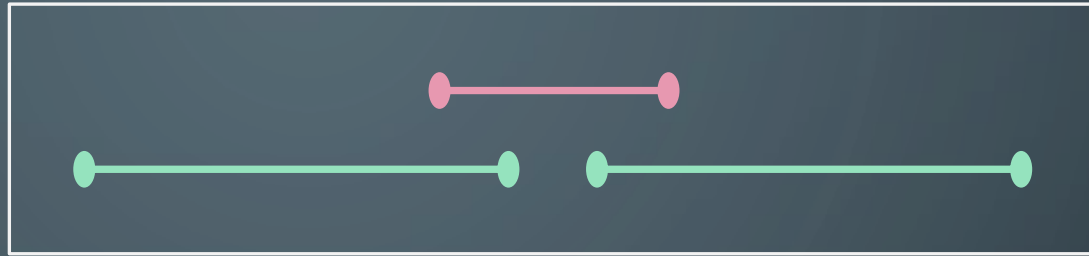
Pick the next compatible activity that **finishes the latest**.

Pick the activity that has the **MOST conflicts** (i.e. overlaps)?

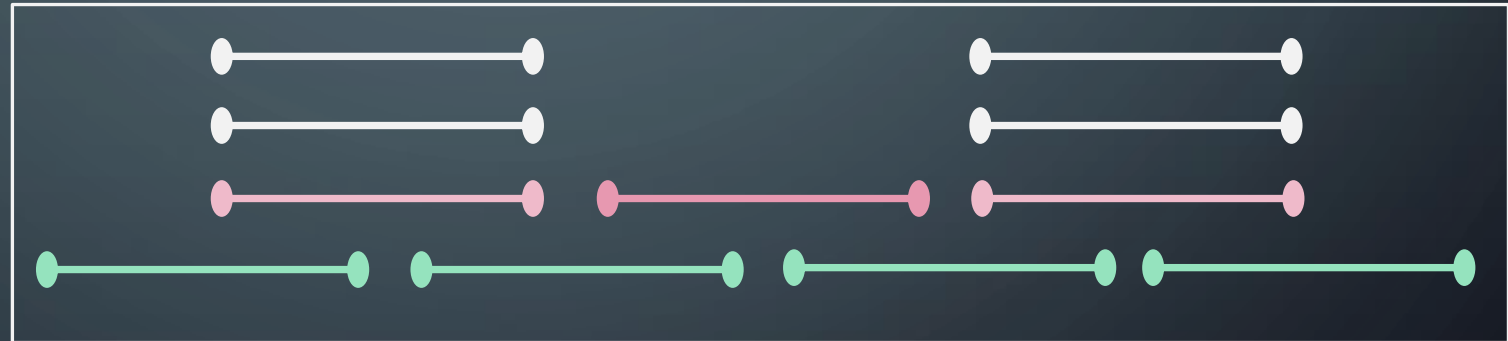
Or something else...

SOME POSSIBLE GREEDY CHOICES

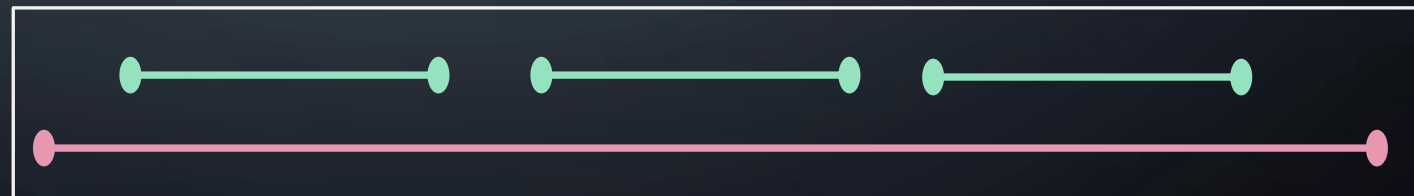
Shortest interval



Fewest conflicts



Earliest start



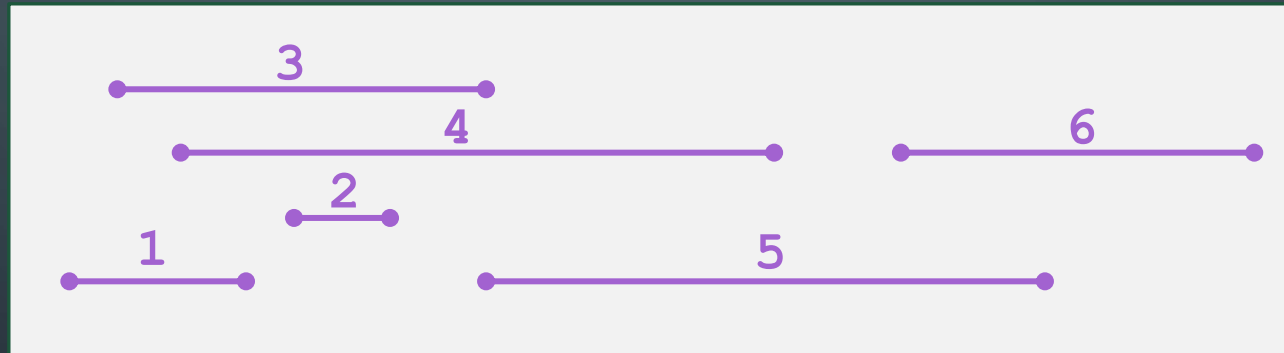
ACTIVITY SELECTION: A GREEDY ALGORITHM

Pick the next compatible activity
that **finishes the earliest**.

Turns out this one is correct. Intuition: This one
maximizes the size of the subproblem!

EXAMPLE

Activity Selection: Given a list of activities $A = \{a_1, a_2, \dots, a_n\}$ where each activity $a_i = (s_i, f_i)$ is defined by a **start time** and **finish time**. Select the maximum number of activities that can be fit into one schedule



What is the optimal solution?

BACK TO SEMESTER AT SEA...

Id	Start	Finish	Len	Activity
1	0	6	7	Fractals, Recursion and Crayolas
2	1	4	4	Tropical Drink Engineering with Prof. Bloomfield
3	2	13	12	Managing Keyboard Fatigue with Swedish Massage
4	3	5	3	Applied ChemE: Suntan Oil or Lotion?
5	3	8	6	Optimization, Greedy Algorithms, and the Buffet Line
6	5	7	3	Hydrodynamics and Surfing
7	5	9	5	Computational Genetics and Infectious Diseases
8	6	10	5	Turing Award Speech Karaoke
9	8	11	4	Pool Tanning for Engineers
10	8	12	5	Mechanics, Dynamics and Shuffleboard Physics
11	12	14	3	Discrete Math Applications in Gambling

Solution: 2, 6, 9, 11

[illegible][illegible]

[illegible][illegible]

[illegible][illegible]

[illegible][illegible]

[illegible][illegible]

RECURSIVE GREEDY ALGORITHM

```
RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)
1   m = k + 1  // start with the activity after last added
2   while m ≤ n and s[m] < f[k]  // find first activity to finish
3       m = m + 1
4   if m ≤ n
5       return {am} U RECURSIVE-ACTIVITY-SELECTOR(s, f, m, n)
6   else return ∅
```

- Initial call: RECURSIVE-ACTIVITY-SELECTOR(*s*, *f*, 0, *n*)
- Run time is $\theta(n)$, assuming the activities are already sorted by finish times

NON-RECURSIVE ALGORITHM

```
greedy-interval (s, f)
    n = s.length
    A = {a1}
    k = 1    # last added
    for m = 2 to n
        if s[m] ≥ f[k]
            A = A U {am}
            k = m
    return A
```

- s is an array of the intervals' start times
- f is an array of the intervals' finish times, sorted
- A is the array of the intervals to schedule
- How long does this take?

A decorative graphic on the left side of the slide, consisting of a network of thin, light-blue lines and small circles, resembling a circuit board or a neural network diagram. The lines are vertical and horizontal, with some diagonal connections, and the circles are small and white, acting as nodes or junctions.

PROVING CORRECTNESS

PROVING THE GREEDY CHOICE PROPERTY?

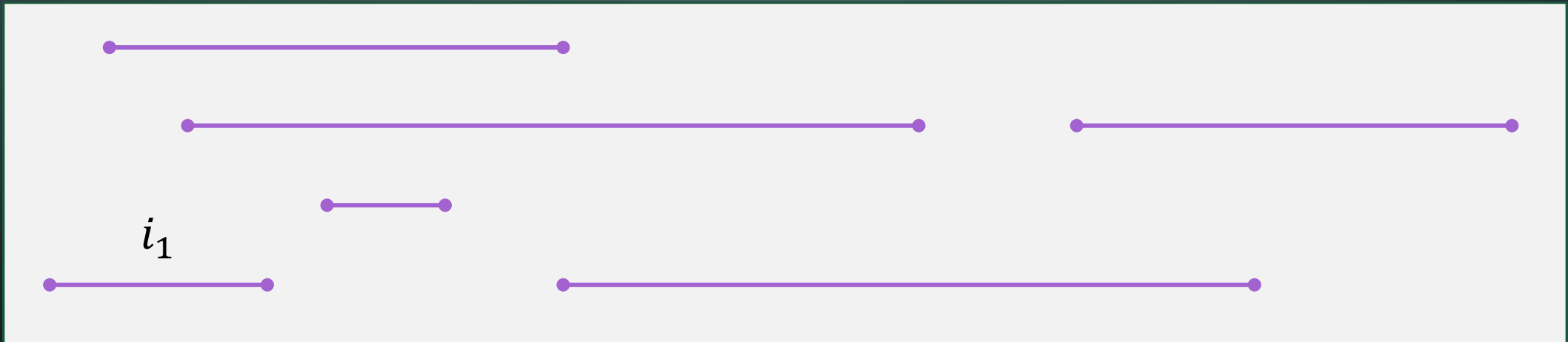
Claim: The activity with the earliest finish time is guaranteed to be a member of some optimal solution

Proof Overview:

- Assume *earliest finish time activity NOT in optimal* schedule
- Describe what *optimal solution must look like* instead
- Show we can *switch in the earliest finish time item* without changing the optimality of the solution (take one thing out, put one thing in)

DOES GREEDY ALWAYS FIND OPTIMAL SOLUTION?

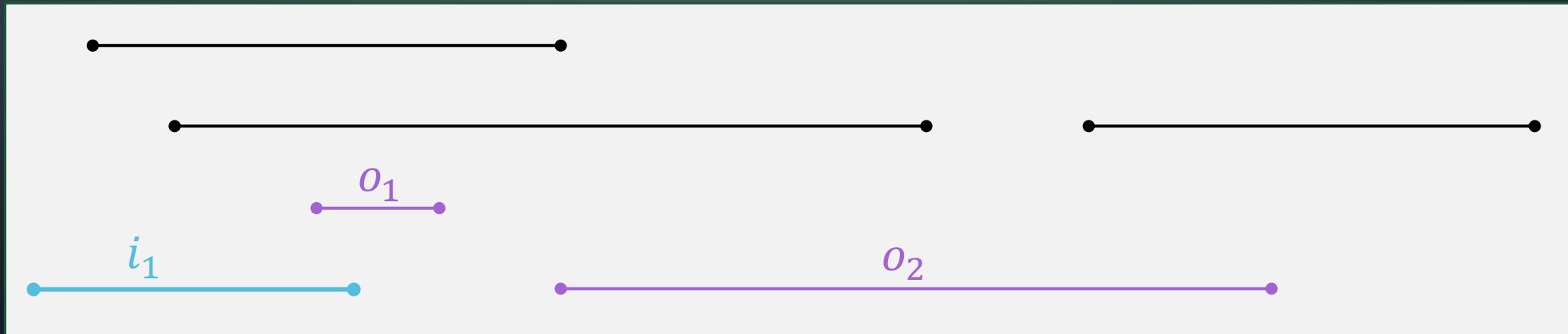
Claim: The activity with the earliest finish time is guaranteed to be a member of some optimal solution



DOES GREEDY ALWAYS FIND OPTIMAL SOLUTION?

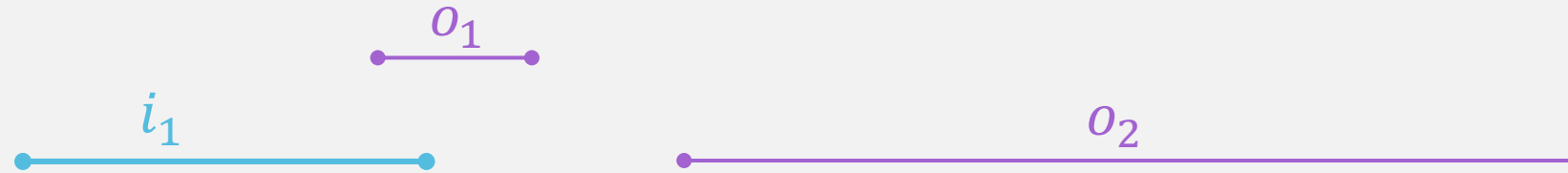
STEP 1: Assume i_1 is NOT in the optimal solution.

STEP 2: If i_1 NOT in optimal solution, then solution contains other intervals instead $O = \{o_1, o_2, \dots, o_n\}$ and $i_1 \notin O$



**Note: This picture is just an EXAMPLE to help you visualize. Not good enough for a proof on its own*

DOES GREEDY ALWAYS FIND OPTIMAL SOLUTION?



Step 3: Let's compare o_1 , o_2 , and i_1 , We KNOW THAT:

$i_1.f \leq o_1.f$ // i_1 finishes before o_1 because greedy selected i_1

$o_2.s \geq o_1.f$ // o_2 must start after o_1 finishes

$i_1.f \leq o_1.f \leq o_2.s$ // combining previous two lines o_2 starts after i_1 finishes

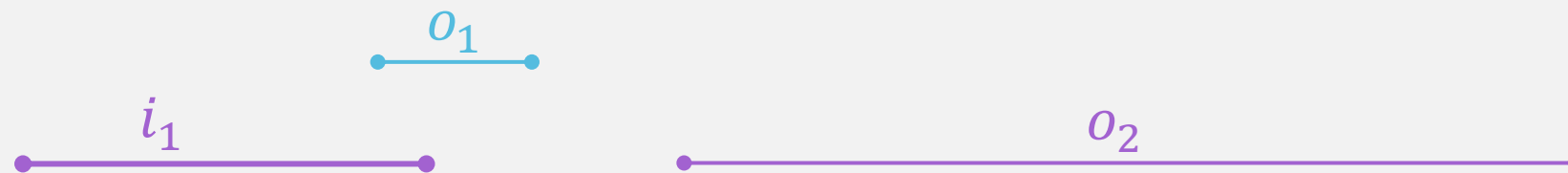
// so i_1 and o_1 can be swapped!

DOES GREEDY ALWAYS FIND OPTIMAL SOLUTION?

From previous step:

$$i_1 \cdot f \leq o_1 \cdot f \leq o_2 \cdot s \quad // \text{ so } i_1 \text{ and } o_1 \text{ can be swapped!}$$

Step 4: Exchange o_1 for i_1 . Solution size doesn't change at all (still optimal) and contradicts our assumption the NO optimal solution contains i_1



Q.E.D. Greedy Choice Property Proven

WHAT DID WE LEARN

- Activity Selection
 - Has many possible Greedy Choices that could be made.
Not obvious which was correct.
- Optimal Substructure:
 - Was a function of changing the time frame within which the subproblem was solved
- Greedy Choice Property:
 - Another example of proving the greedy choice property!